

# Teaching Students Global Software Engineering Skills using Distributed Scrum

Maria Paasivaara\*, Casper Lassenius\*, Daniela Damian†, Petteri Rätty\*, and Adrian Schröter†

\*Software Process Research Group, Aalto University  
FI-00076 Aalto, FINLAND  
firstname.lastname@aalto.fi

†SEGAL, University of Victoria  
Victoria, BC, Canada  
danielad@cs.uvic.ca, schadr@acm.org

**Abstract**—In this paper we describe distributed Scrum augmented with best practices in global software engineering (GSE) as an important paradigm for teaching critical competencies in GSE. We report on a globally distributed project course between the University of Victoria, Canada and Aalto University, Finland. The project-driven course involved 16 students in Canada and 9 students in Finland, divided into three cross-site Scrum teams working on a single large project. To assess learning of GSE competencies we employed a mixed-method approach including 13 post-course interviews, pre-, post-course and iteration questionnaires, observations, recordings of Daily Scrums as well as collection of project asynchronous communication data. Our analysis indicates that the Scrum method, along with supporting collaboration practices and tools, supports the learning of important GSE competencies, such as distributed communication and teamwork, building and maintaining trust, using appropriate collaboration tools, and inter-cultural collaboration.

## I. INTRODUCTION

Global Software Engineering (GSE) is now an established trend in software development, and so are agile methods, such as Scrum [1] and XP [2]. Due to their emphasis on face-to-face communication, agile methods seemingly present a practical dilemma for distributed development for which communication is one of the greatest challenges. However, with their equal emphasis on frequent communication, transparency of progress and short iterations, they in fact provide an effective approach to address challenges in distributed collaboration, especially when augmented with GSE best practices [3]. Adopting agile methods in distributed settings seems to be a rising industry trend, as agile methods both provide several benefits, as well as strategies for addressing GSE challenges [4], [5]. Therefore, software engineering curricula must respond by providing students with knowledge and experiences of globally distributed agile projects. Furthermore, as software engineering researchers and educators, our community should increase its efforts in evaluating methodologies for teaching GSE (e.g. [6]).

The number of reported experiences on teaching GSE has steadily grown in the last decade [7]. Most GSE courses are practical, project-driven and arranged in collaboration between two or more universities, enabling students to learn GSE in realistic settings [8]. However, most courses so far have used

the waterfall development process in the distributed course projects, with the exception of Scharff et al. [9]–[11], who report positive experiences from the adoption of Scrum, having previously used the waterfall model. Implementing distributed Scrum in classroom environments requires additional technological support for the frequent synchronous interaction across sites and is far from trivial. The Scrum implementation reported by Scharff and colleagues had several discrepancies compared to industrial best practice for distributed Scrum. For example, they did not stress synchronous communication or exclusively use cross-functional teams, and had a single team of up to 20 students. The authors do not report any systematic data analysis of learning of GSE competencies either.

In this paper we report on a richer implementation of distributed Scrum in teaching GSE, as well as our assessment of learning of GSE competencies such as distributed communication and teamwork, building and maintaining trust, using collaborative tools, and inter-cultural collaboration. We believe that distributed Scrum is both a methodology with which our graduates should become familiar and an important paradigm for instructional design in teaching these critical GSE competencies. As Scrum has short sprints with several formal meetings for Sprint Planning, Demos and Retrospectives, as well as emphasizes early and frequent communication within and between teams, as well as with the Product Owner (PO), it gives the students the possibility to experience the whole life-cycle of a distributed project, while reflecting on and continually improving their team working practices.

Following a review of related work in Section II, we describe our instructional design of a GSE course between two universities located in Finland and Canada in Section III. Our distributed Scrum implementation was based on known best practices in industrial agile GSE, such as frequent synchronous communication between the sites, the use of videoconferences for the Daily Scrums and Sprint Planning and Retrospective meetings, sharing the pain of time-zone differences, and remote site visits. To assess learning of GSE competencies and the role played by distributed Scrum in our instructional design we employed a mixed-method research methodology as described in Section IV. We used pre-course, post-course,

and sprint surveys to collect data on communication, teamwork and trust levels, as well as tracking of online communication in email and chat channels, analysis of videoconferencing project meetings and a series of post-course interviews. As evidence of learning, we describe the students' reported challenges in their distributed collaboration and teamwork, strategies to overcome them, as well as the effectiveness of these strategies.

## II. PREVIOUS WORK

### A. Global Agile Development

Even though agile methods and GSE seem to be incompatible, as agile methods emphasize frequent face-to-face communication, while communication is the biggest challenge in GSE, global agile development seems to be a rising industry trend [4], [5]. Two main reasons have been given for the popularity of agile methods in GSE: their general popularity in industry, and the fact that they address the challenges following from a distributed way of working [5].

Scrum, one of the most popular agile methods in the industry, has been shown to fit GSE projects well [4]. Since one of the biggest challenges associated with GSE is lack of communication, Scrum helps by emphasizing communication and providing a structure for frequent formal communication both within and between the teams, as well as with the PO. This formal communication includes brief daily status meetings, called Daily Scrums, for each Scrum team, and in the case of several teams working in the same project, Scrum-of-Scrums meetings for inter-team coordination. Moreover, Scrum introduces Sprint Planning meetings in the beginning of each sprint to plan and estimate the new user stories (requirements) to be implemented, as well as Demos and Retrospectives in the end of each iteration to showcase newly implemented functionality and to improve the process [1].

Research on the usage of Scrum in distributed projects has found these formal meetings to force team members between sites to communicate frequently, and to facilitate informal communication after the meetings, as participants notice that there are issues needed to be solved or questions to ask [3].

Although lack of synchronous communication due to time-zone differences can be a real problem for distributed projects, a systematic literature review [4] reports strategies for dealing with the challenges such as synchronized working hours, modified Scrum practices and a rich communication environment with sufficient tool support.

### B. GSE Teaching

A systematic literature review on GSE teaching [7] shows that the number of reported experiences on teaching GSE has steadily grown since the beginning of this century. The review stresses that GSE teaching must be supported by practical experiences through which students can learn by doing. Indeed, this also seems to be the case in practice: a systematic mapping study of 19 GSE courses [8] found only one course that was theoretical in nature, while all others had a distributed project as a central element.

As most reported GSE courses were still using a waterfall process, in contrast to the popularity of agile methods in the industry, there seems to be a discrepancy between industrial and academic practice. Moreover, only a subset of the reported courses teach students the whole life-cycle of a software development project. In fact, the systematic literature review [7] concludes that it is not possible for instructors to cover all of the stages of GSE and therefore one specific aspect should be focused on. As evident from this paper, we do not share this view.

Only a few courses have been run for several years during which the course organizers have collected experiences and improved their courses. A GSE course distributed between Croatia and Sweden [12], [13], arranged yearly since 2003, is one of the longest lasting GSE courses. The course has around ten projects yearly, distributed between two sites with 6–8 students working for each project. The DOSE course [14], organized since 2007, is one of the largest GSE courses in the number of participants: during 2010 it was organized between eleven universities, from ten countries distributed to Europe, Asia and South America, had 110 students working in eleven groups, each distributed between three sites, all groups contributing to a single software system.

### C. Agile Methods in GSE Teaching

We were able to find only one GSE project course that reportedly had used an agile method [9]–[11], describing their experiences on teaching a GSE course using Scrum practices over three years with positive experiences, after teaching the same course using waterfall for four years. According to them Scrum increased the transparency of both the process and the developed software product.

This course had students from three to four countries: US, India, Cambodia and Senegal forming one to three Scrum teams. After testing different set-ups for the PO and SM roles, the authors recommend giving both roles to persons external to the teams to provide a real Scrum experience. The regular Scrum meetings, that are normally conducted face-to-face, were mainly organized in written format, using chat or wiki. The authors do not report how this primarily written communication affected collaboration success.

Overall, Scharff et. al [9]–[11] report good learning outcomes. Their students had found Scrum to be a demanding process as it requires a frequent and regular pace of work, which the students had not always maintained, e.g., by skipping some meetings or not doing the required preparations. The authors show that teams that conducted more Scrum ceremonies developed a better solution in terms of implemented and accepted user stories.

### D. Skills Needed for GSE

Communication is the single biggest challenge of GSE, whereas other challenges, e.g., cultural, language and background differences, lack of overlapping working hours, lack of frequent contact, lack of trust and lack of willingness to communicate openly [15] are all related to communication.

Most GSE courses report teaching skills for coping with these challenges. Specific challenges reported include knowledge backgrounds [12], communication problems [12], [14], language difficulties [12], [14], social and inter-cultural issues [12], and motivational issues [14].

The GSE teaching literature has identified skills that GSE professionals need and are thus important to be taught to students learning GSE (e.g. [7], [16]): regular communication with distributed team members [16], [17], team dynamics [16] working in culturally diverged teams [17], managing time [17], and using collaborative technologies [17]. Notably, all of these are soft skills. Product architecture is another important aspect of GSE since, arguably, modular architectures offer the potential to reduce cross-site communication [18]. However, the evolving nature of designs over the course of a project limits a design's ability to predict and optimize project cross-site communication [19]. The need to respond to changing architectural dependencies further emphasizes the importance of developing communication skills in our GSE courses.

GSE courses are often positioned for the final years of studies, as capstone courses [12], thus the students get both a possibility to apply the technical skills they had learned during their studies in a close-to-real-project context, as well as learn "soft skills" that are crucial for GSE projects, but needed for all kinds of software development projects. Thus, we conclude that GSE courses, besides giving a possibility to apply technical skills, should especially concentrate on teaching soft skills.

### III. COURSE INSTRUCTIONAL DESIGN

#### A. Learning Goals

Our main goal was to teach GSE competences by emulating a modern real-world GSE environment through the use of agile practices that involve significant inter-team, inter-site and inter-cultural communication, as well as to teach and expose students to various collaborative technologies that provide infrastructural and cognitive support for effective communication in global teams.

Based on the reviewed GSE literature, we have listed the main GSE competences that we wanted to teach as the following:

- Distributed communication
- Distributed teamwork
- Building and maintaining trust
- Usage of communication and collaboration tools for distributed work
- Inter-cultural collaboration

#### B. Course Setup

We summarize the course setup here, and a more detailed description of the course set-up can be found in [23]. The course was run in collaboration between the University of Victoria, Canada and Aalto University, Finland, over a 3-month period, from January to April 2012, with slightly different configurations at the two locations. Whereas the Aalto University course was run as a capstone, project-based

course in which students attend no lectures but fully worked on the project, the University of Victoria course setup included a more traditional structure in which students participated in lectures, as well as in-class and online discussions of experience. The Finnish course started in September 2011, thus the Finnish students had already worked for a few months before the Canadians joined in January 2012.

In total the course had 25 PhD, Master's and undergraduate students, 16 in Canada and 9 in Finland. The students worked on a single large project of which goal was to extend Agilefant (<http://www.agilefant.org>), an existing free backlog management system developed at Aalto University.

Students were organized into three distributed Scrum teams, each consisting of 7-8 members (4-5 Canadian students and 2-3 Finnish students). The Product Owner (PO), a staff member at Aalto University, as well as the Scrum Master (SM) were located in Finland. The SM was an experienced student, who worked as a SM for all the teams, and thus had a crucial role in inter-team co-ordination.

Between the universities there were slight deviations in physical setup. The Canadian students had a war room containing a large common area that was equipped with multi-feed cameras and five smart boards to be used as video conferencing interfaces. Additionally there were three adjacent separate offices for private or ad-hoc meetings and other development-related purposes. The Finnish war room was an open office space with several computers, but no dedicated equipment.

To follow the GSE best practices [24], two visits were organized to kick-off the project and enabled the instructors and students to get to know each other and their cultures. First, the Canadian instructor visited Finland for a one day meeting with the Finnish team in December 2011. Subsequently, in January 2012, when the Canadian course commenced, the Finnish instructor together with the PO and two students with extensive technical knowledge of the project and agile practices (one of them the SM) visited the Canadian site for nine days. During the visit, Scrum training, as well as technical training on Agilefant was given. The Canadians also performed one actively coached six-day mini-sprint to familiarize themselves with both the Scrum practices and the Agilefant codebase.

#### C. Distributed Scrum and Collaboration Infrastructure

In this section we describe in detail the Scrum practices that we imposed to support the learning of the GSE competencies described earlier. Their mapping to these competencies is provided in Table II.

We used two-week sprints, resulting in the project having five sprints. Each sprint started by a synchronous Sprint Planning meeting for all three teams organized via two-way Google Hangout videoconferencing session, between the two war rooms. First, the Finnish PO presented an overview of the user stories to be developed in the sprint, as well as an initial allocation of user stories to the three teams. Then, the teams split into three separate videoconference sessions to estimate the stories, and perform an initial task breakdown.

TABLE I: Data Collection and Analysis

| Data collection    | Purpose / Instruments                                           | Data collected                                                                  | Analysis |
|--------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------|----------|
| Pre-course survey  | Student background, expectations                                | 25 responses                                                                    |          |
| Sprint surveys     | Trust [20], Teamwork [21], [22], Communication                  | 19-22 responses                                                                 |          |
| Emails             | Communication                                                   | 4210 emails                                                                     | SNA      |
| Chat               | Communication                                                   | 2859 messages                                                                   | SNA      |
| Meeting videos     | Communication                                                   | 31 Daily Scrums, 2 Demos & Sprint Planning meetings                             |          |
| Observations       | Teamwork, communication                                         | 5 Demos & Sprint Planning meetings, Retros, Daily Scrums, teamwork in war rooms |          |
| Student logs       | Communication, encountered issues, tasks                        |                                                                                 |          |
| Agilefant          | Estimation accuracy, work breakdown                             | Task estimates and actuals, burndown                                            |          |
| Post-course survey | Learning                                                        | 20 responses                                                                    |          |
| Interviews         | Learning, communication, communication tools, improvement ideas | 13 interviews, 40-90 min each (8 Canadian & 4 Finnish students, Finnish PO)     | Atlas.ti |

The PO connected with each distributed team, one at a time, and explained the user stories in more detail, giving the team a basis for story-point estimation. Immediately after these team-specific sessions, each team communicated their plan to the PO and each other in another joint videoconference session.

During the sprints, teams held two weekly 15 min videoconference Daily Scrum meetings, followed by informal hangout sessions to discuss project-related issues or strategies for the given sprint. Since the time allocated to this project was 10–12h/week/student, this frequency for Daily Scrums was determined as adequate. The Daily Scrum meetings were scheduled by each team separately and took place on Google Hangout; typically in early morning at one site, which was late evening at the other site. For example, one team always had its Hangouts at 9:00 AM in Finland and 11:00 PM in Canada.

At the end of each sprint, each team had their Retrospective meeting over Google Hangout to reflect and improve their ways of working, followed by a joint Demonstration to the PO and the other teams in the two connected war rooms. The Demos, Retros and Sprint Planning meetings all took place every second week during a time-boxed 2.5h slot. The order of the team specific Sprint Planning sessions and Retros varied team by team, so that the teams could use the PO's time efficiently.

#### IV. METHODOLOGY

To assess the role of distributed Scrum in learning, we collected data using a mixed-method approach combining qualitative and quantitative methods [25]. We used surveys, observations, videorecordings of meetings, collection of communication data (emails, chats), and interviews (by a course external researcher), and triangulated results whenever possible. For each of our learning goals (the GSE competences) we sought to identify the students' expectation of learning, experiences, as well as reported actual learning. A summary of data collection instruments and analysis methods used is shown in Table I.

#### V. RESULTS

In this section we describe how distributed Scrum, augmented with a number of best practices in GSE, enabled the learning of the GSE competencies in our course: distributed

communication and teamwork, building and maintaining trust, usage of collaboration tools, and inter-cultural collaboration. Specifically, we discuss the students' reported learning outcomes as a result of using distributed Scrum and supporting GSE practices in our instructional design (summary in Table II).

##### A. Distributed Communication

Our first goal was to teach the students strategies for successful distributed communication, thus we will first look at how the student teams communicated. Figure 1a shows the social network of the entire project, based on an analysis of all email and chat communication. Each node represents one student, and the borders for teams t0, t1 and t2 are indicated by the solid lines. The PO and SM form a "team" of their own. The size of the nodes show betweenness centrality, a measure that can be used to assess the "importance" of a node in a social network [26], and the color the clique the node belongs to. The thickness of the edges indicate the amount of communication between two nodes.

A few things are worth pointing out. First, a clique analysis [27] identifies exactly the three assigned distributed teams and does not show, e.g. site-specific cliques. Thus, it seems that the distributed teams really worked as teams and that the geographical distribution did not create notable boundaries. Second, the role of the SM and PO are important, as their high betweenness centrality indicates. According to Scrum, their roles should be central and include a lot of communication, which seems to be the case here. Third, the picture shows that the three teams had different strategies for communication and different leadership styles. In teams t0 and t1, about half of the teams are closely knit, with the other members being more peripheral, whereas team t2 had a clear team leader. According to Scrum the team members should have equal roles. Thus, based on the email and chat communication it seems that teams t0 and t1 are close to that ideal. However, team t2 had a clear emergent leader.

In the next sections we discuss the students' reported learning of distributed communication skills. As shown in Table II, the supporting Scrum practices for this competence were Scrum's short sprints, Daily Scrums, Spring Planning meetings, Demos and Retrospectives, continuous communica-

tion and frequent interaction with the PO. We also provided a wide range of tools to support both formal and informal communication.

1) *The Importance of Early and Frequent Communication:* Our students learned by trial and error the importance of frequent communication and communicating early and instantly:

*When you are in a hurry you easily start lowering the amount of communication, when you feel that it is something that could be dropped out. But I noticed very quickly that this was not the case. Also when in schedule pressure you need to keep communication in your mind, that it is important, that things do not proceed, if you do not allocate time also for communication.*

— A Finnish student

*Communicating early and instantly. So, not to wait at all, if there is any sort of hiccup in the plan, or if anything happens or you have to delegate any work, you have to make sure you communicate right away.*

— A Canadian student

All teams used Daily Scrums twice a week as videoconferences in Google Hangout. These meetings created awareness on what was happening in the project and which topics needed further discussion. They helped the distributed team members get familiar with each other, thus lowering the barrier to initiate communication, and were found as the most useful form of communication:

*I really liked how we did the stand-ups (...) the stand-up is really good in just communicating what people are doing right now, what have they done, if anybody's in trouble. And as simple as they are, they're really good.*

— A Canadian student

*The stand-up meetings in Hangout so that everybody could see each other were great. (...) That you could see each other was cool. (...) You can read and interpret people, their opinions better than when only hearing their voice. Especially when there are some language difficulties, it helps to understand, communication is easier when you can see the others.*

— A Finnish student

2) *Finding the Appropriate Amount, Style and Tools for Communication:* Scrum's short sprints, need for frequent integration of work and maintenance of awareness of progress

made students experiment with numerous communication tools. Although overwhelming at first, students settled on using certain tools depending on the purpose of the communication. When email was the main communication medium it became overused, resulting in the teams soon replacing part of email communication with other media better suited for their needs.

*Well, we definitely learned which tools were the best to communicate. (...) So, at the beginning we had I think six or seven tools to communicate, and we learned that less is more, so we actually converged down to about four methods. (...) our communication definitely improved during the later sprints.*

— A Canadian student

Despite the large time-zone difference, the students preferred synchronous communication over emails whenever possible: Google Hangout for short videoconference meetings for pair and group work; chat discussions for short questions and agreeing on time slots for face-to-face or videoconference meetings; and Google Docs and Agilefant, the backlog management tool, for creating awareness for ongoing or completed work.

*I'd say that [the Google Hangout] was probably the most effective mode of communication, especially for conversing with the Finns. It's quite more effective to actually talk to somebody than to go through email. You kind of go through email if it's last resort or you know they are asleep right now.*

— A Canadian student

The Finnish students had agreed to sit together for a couple of days every week, during which most of them would be working in the same room at the university. In the interviews some of the Canadians mentioned that they should have adopted the same practice, since coding together in the same room would have been more fun than coding alone. Even though everybody might be working on their own tasks, there would be other team members around to ask questions, exchange ideas and it would create a feeling of teamwork. This practice is something that we will certainly recommend to our students in the next course.

Being precise and unambiguous in written communication is equally important and challenging. The students mentioned

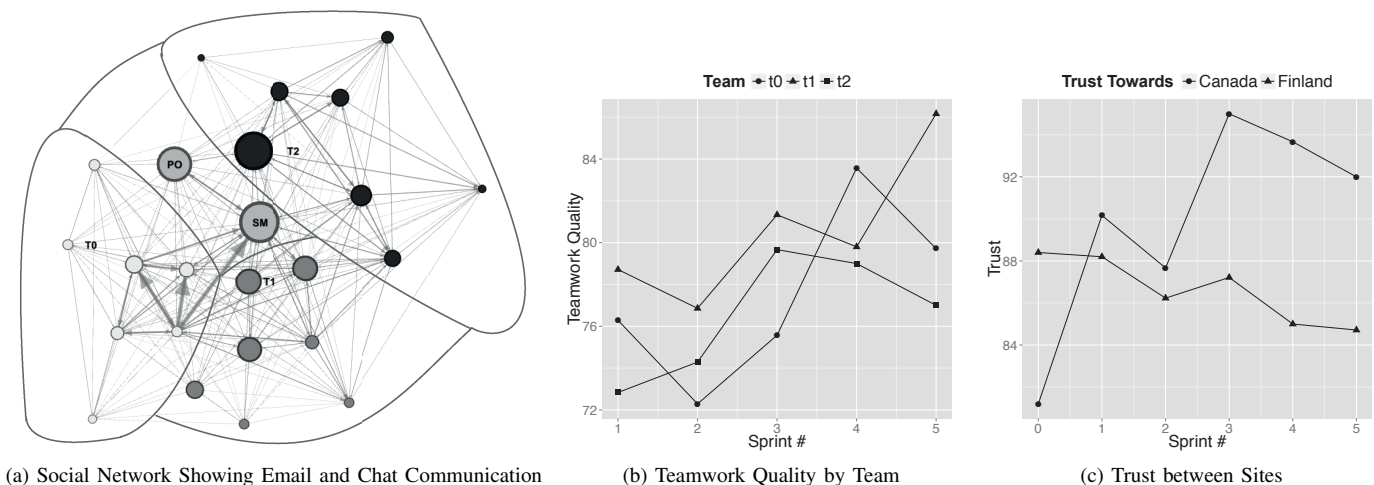


Fig. 1: Social Network, Teamwork Quality and Trust during the Sprints

that they learned how to improve the style of communication, by being precise in all communication and in particular as explicit as possible in asynchronous communication.

*I learned about the communication, how we should spell out everything to them [to the Canadian students] and they have to spell out everything to us. It is so easy to get misunderstandings.*  
— A Finnish student

*With regard to asynchronous communication you gotta make it more effective, so that you are not going back and forth, which between different time-zones can take days. You wanna limit, clarifications and such, just one email at most, get it all in there, be very direct and try to be as explicit as possible.*  
— A Canadian student

3) *Communicating with a Customer*: Scrum relies on on-going communication with the PO and the students' comments indicate their learning of the importance of frequent communication with the customer. In this project the PO participated in all Demos and Sprint Planning meetings and switched from team to team during the second phase of the Sprint Planning. The PO's enthusiasm for the project and availability for questions and ad-hoc meetings whenever he was awake was motivating:

*He was very enthusiastic about the project, which was good. He sort of encouraged a team spirit and motivated us.*  
— Canadian student

We observed that the students' communication behavior with the PO changed during the course. Due to their learning they became more likely to initiate contact and ask questions of the PO. The students quickly learned that they did not have to limit their communication with the PO to only formal meetings (Sprint Planning meetings and Demos). Instead, many of the groups started to arrange informal demos to the PO during the iteration through Google Hangout, whenever

they got something ready to show him for immediate feedback. The formal demos of their stories became more efficient. Adapting to the large time-zone difference, one team initiated an asynchronous feedback cycle by sending screenshots by email for commenting, if the PO was not online.

*I personally felt that our team should have taken more of an initiative to talk to him [PO], rather than the other way around. (...) I really think that any problems that we had with [the PO] was failing from our side to communicate, not necessarily from him. I don't personally feel that a Product Owner should be keeping us on task and asking us about progress necessarily. The Product Owner, he's supposed to be busy, he's supposed to be out there. It's our end that need to communicate.*  
— A Canadian student

Scrum requires that each team scopes the work at the beginning of each iteration, and this has not been without challenges. The students quickly learned that in a real project the customer may have too many requests and lack a prioritized ranking. Our students' ability to scope the first iterations was low. The teams had to learn to decline when the PO asked for too many user stories in an iteration, and ask him to prioritize whenever everything planned could not be done:

*Well, [the PO] overloaded us during the sprints, but that was partially our fault for not budgeting or scoping things properly. (...) when we are overloaded during a sprint and it's clear that the story needs to be deferred, we need to take the initiative and talk to [PO] and ask him what we should do.*  
— A Canadian student

*[the PO] definitely pushed us, like we worked a lot and got a lot of stuff done, then he'd have more for us next sprint, and that's I think a good thing. If you need to defer something or say you have too much work, then you'll have to actually tell him. Otherwise, you'll just keep getting more work. (...) So the team really has to say when*

TABLE II: Student Reported Learning Outcomes as a Result of using the Distributed Scrum and Supporting GSE Practices

| GSE Competence                           | Supporting Scrum Practices                                                                                                                | Supporting GSE Practices                                                                                               | Learning Outcome as reported by students                                                                                                                                                                                                                         |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Distributed communication                | Short sprints, Daily Scrums, Sprint Planning meetings, Retrospectives, Demos, continuous communication. Frequent interaction with the PO. | Providing a wide range of tools to support both formal and informal communication: chat, email, videoconferencing etc. | (1) The importance of early and frequent communication. (2) Finding the correct amount, style and tools for communication. (3) The importance of frequent communication with the PO and strategies to initiate contact and ask questions.                        |
| Distributed teamwork                     | Frequent synchronous communication (Daily Scrums, Sprint Planning meetings, and Retrospectives)                                           | Cross-site teams. Distributed pair programming.                                                                        | (1) Why and how to create awareness in a project. (2) Strategies for creating awareness of work asynchronously. (3) Strategies to share the pain of time-zone differences. (4) Strategies for time management. (5) How to benefit from distributed pair work.    |
| Building and maintaining trust           | Frequent deliveries of code. Continuous communication: Daily Scrums. Transparency of work and work quality.                               | Giving faces to both sites by travel to remote sites. Distributed pair programming.                                    | (1) The importance of getting to know each other (especially remote colleagues) in the beginning. (2) Trust building through seeing good quality work. (3) How lack of quality work damaged trust in remote colleagues. (4) Maintaining trust through awareness. |
| Tool usage for distributed collaboration | Daily Scrums. Frequent integration of code (version control and build systems). Backlog tool.                                             | Providing a large selection of tools for collaboration.                                                                | (1) Experience with a good number of collaboration tools. (2) Being able to select a good combination of tools, after having been overwhelmed by too many tools. (3) Selecting the suitable tool for particular purposes in collaboration.                       |
| Inter-cultural collaboration             | Daily Scrums, Retrospectives.                                                                                                             | Giving faces to both sites by travel to remote sites. Cross-site teams. Distributed pair programming.                  | (1) Understanding the differences in communication styles and personalities across the two countries. (2) The importance of frequent collaboration and personal discussions across countries to lower cultural barriers.                                         |

*it's enough. (...) we definitely learned that you have to be transparent and very responsive to the Product Owner.*

— A Canadian student

## B. Distributed Teamwork

In Scrum, small—normally 8–12 person teams—are self-directed and cross-functional, i.e., all team members may do all kinds of tasks and the team members together decide the team's internal way of working. In our course the students reported that, due to Scrums' prescribed frequent synchronous communication, they had to figure out how they could work efficiently as a team despite the geographical separation and the 10-hour time-zone difference. The teams had innovative ways to create awareness, strategies for coping with the time-zone differences and for managing their own time, as well as introduced pair work, both for distributed and collocated pairs.

Figure 1b shows the data we collected on the teamwork quality indicator [22] after each sprint. Although all the teams had some ups and downs in their teamwork, the overall trend is upwards with all teams reporting higher values in the end, than they had in the beginning, indicating that in spite of some difficulties the teams were able to improve their teamwork quality during the project. Interestingly, if we compare the communication structure of the teams (Figure 1a) to teamwork quality, we can see that team t1 which had very high scores for teamwork quality, also had the most egalitarian communication structure, suggesting that the team communicated as a true Scrum team should.

1) *Creating Awareness*: Daily Scrums were very much appreciated by the students which indicates their enhanced appreciation of maintaining awareness in the project.

*I think that the need for as close communication as possible between the two sites was an interesting lesson. (...) Just the importance of sort of keeping everyone on the same page. (...) making sure that both sides were, just have the same vision of the project and were doing the same thing, and knew what people are doing. (...) it got sort of difficult to tell exactly where the other side was at sometimes, if they hadn't checked in their code.*

— A Canadian student

Although the Daily Scrums were scheduled twice a week, students remarked their added benefits in improving the awareness and in encouraging the students to work daily:

*If we could somehow have actually Daily Scrums, that would force people to at least look at their project every day.*

— A Canadian student

*To my surprise I noticed that it [Scrum] somehow works also in this kind of an environment. (...) I did not believe that the stand-ups twice a week would correspond the normal Daily stand-up, well, it did not, but it worked, our team was able to keep aware of what was happening.*

— A Finnish student

Due to the mostly asynchronous nature of development work in the distributed teams, students working at different times found it important to know exactly what others in the same team or pair had been doing when starting their own working pass. Thus, one of the teams decided in a Retrospective to introduce Google Docs for briefly reporting the things each had done during their last working pass, as well

as for asking questions and for help. The students reported that this had improved the situation.

*(...) we created this sprint communication log, which is a Google Doc, where you can update your progress, you can ask for help and people can reply you there. (...) The idea of the communication log was that we would have a persistent status, and if we had a question it would be persistently there.*

— A Canadian student

The backlog management tool Agilefant, that the teams were developing, were used by the teams in their own work as well. The students found it as a very good tool for creating awareness both between the teams and inside one team, since there everybody could easily see what tasks had been finished and what were ongoing tasks and who was working with what.

*I thought that Agilefant did a good job of communicating the status of the stories, to see where the groups are in terms of burndown.*

— A Canadian student

2) *Coping with Time-zone Differences and Strategies for Time Management*: The Canadian-Finnish collaboration imposed a challenging 10-hour time difference between the students in each team. They learned that arranging common meeting times and synchronous working slots for pair/team work, especially across this large time-zone distance, required good time management. Several students mentioned that during this course they had learned to manage their own time much better, positively influencing their ability to estimate, schedule, and allocate tasks in the short sprints.

*I think the technical things I could have learned just on my own (...) but I think these communication and time management, and these sort of lessons required a project like this to learn, because there's things that you wouldn't really understand that you need to work on, until you actually have problems with them.*

— A Canadian student

*I think it is how to manage time working with teams that aren't all in the same place. (...) your team members in Finland are only gonna be limited hours awake, so if you wanna make the most of synchronous communication you gotta actually be online to talk to them.*

— A Canadian student

Having to schedule common meeting times for the informal distributed meetings resulted in students considering meeting times so that sometimes Canadians had to wake up early/stay up late and sometimes the Finns.

*You have to try to be available and you have to try to see what you can do to make things easier for everyone else.*

— A Canadian student

Even though both the students and teachers tried to be fair in this, the pain was not always shared equally. The Finnish students noted that the timing of the combined Sprint Planning/Demo/Retro meeting was so much easier for the Canadian students, since these meetings took place during their regular class hours in the morning, but for the Finns it was always late evening.

3) *Pair Work*: All student teams used pair programming, an agile practice from XP [2], quite frequently. Pair programming was encouraged by the teaching staff, since there were significant technical knowledge differences within the teams. Thus, by pair work they could even out these differences and

share knowledge. Some of the pairs stayed the same during the whole course and some exchanged pairs after finishing a task or a sprint. A couple of teams used also pair programming with distributed pairs. To our surprise, this ended up being a practice that received very positive comments and the students wished they had used it even more, despite the challenges created by the time-zone difference.

*(...) maybe we should have mixed people so that we would have given more user stories to global pairs. Even though it would have taken longer, there would have been much more communication [between the sites].*

— A Finnish student

The students had agreed on with the pair the times they would be working together, such as a number of two-hour time slots during each week. The distributed pairs either used screen sharing or worked synchronously from their own computers and communicated whenever needed. For example, one distributed pair worked at 3 am/1 pm in Canada and Finland respectively.

*(...) we did quite a lot Google Hangout programming and it worked out surprisingly well. (...) With [a Canadian student] we did all programming, or maybe 70% in Hangout, so that the other ones screen was shared and then we looked together how to implement (...) It worked almost like you had pair programmed with the same computer.*

— A Finnish Student

The students commented that with their pair programming pairs they had personal discussions, which both helped them get to know each other, as well as lowered the cultural boundaries. Several of our interviewed students appreciated the moments when some of their fellow students had shared personal stories that they would remember for a long time. Several students even commented that the most valuable experience during this course was getting to know and working with new interesting people across the globe.

### C. Building and Maintaining Trust

For global teamwork to be possible, building and maintaining trust between the parties is essential. The practices that supported the building and maintaining trust (as shown in Table II) were Scrum's frequent integration of code, continuous communication and the transparency of work, complemented by the travel to the remote site that put a face to the names. The students reported the following 'practices' in building and maintaining trust:

**1) Getting to Know Each Other:** To build an initial sense of trust we followed GSE practices [24] by arranging two face-to-face visits: the Canadian instructor's one-day visit to Finland and the Finnish key person's 9-day visit to Canada. Both visits were highly appreciated by the students, especially the Finn's longer trip to Canada. This trip created good contacts and gave the Canadians a kick-start for the project.

*I think that when we were there, they got a good picture of the Finns and it also lowered the barrier [for the Canadians] to communicate with other Finns. Thus, even though they had not met everybody, in a way they had now some kind of contact surface to Finland.*

— A Finnish student

Thus, the benefits of the Finns' visit to Canada extended to the other Finns who could not travel. Besides the face-to-face visits, many students mentioned the importance of informal communication on non-course related topics that helped them to get to know each other especially across countries:

*I feel like I only really connected to people on our team when I started to talk to them, not necessarily about work, but you know about their lives, connect to them as people. I kind of talked to [a Finnish student] for a bit. He would talk to me about things like his experience in the army and stuff and that was kind of neat.*

— A Canadian student

However, some found it difficult to do it over the communication tools, but not impossible:

*I always felt like I shouldn't be disturbing the people in Finland with really minor details. When I think in hindsight, that's a bad way to approach it. Because I think that also helps to build the trust just sending off little emails in the beginning and not necessarily worrying about.*

— A Canadian student

**2) Trust Building through Seeing High Quality work:** The role of high quality work in building trust among the software developers is well documented in software engineering literature, e.g., [24]. The short sprints enabled our student teams to track everyone's contributions and quality of work early in the project. Students quickly learned to appreciate the team members who did good work and finished what they had promised, and took collective pride over the work done:

*I just liked working with the Finnish team members. They were nice, and they were good at getting things done.*

— Canadian student

*I think it was nice how our team sort of jelled really well. (...) And we all sort of took a collective pride in the quality work that we were doing. (...) That was the most positive aspect of the course.*

— A Canadian student

The reverse effect was also observed in some teams: when programming difficulties resulted in some members underperforming, the team began losing trust in them on important tasks. This trend can be seen in the trust curves in Figure 1c. We can see that the initial trust of Finns towards Canadians was pretty low in the beginning, but it quickly went up already during the first sprint. On the other hand the initial trust of Canadians towards the Finns was high in the beginning, but it started to slowly decline. Our interviews indicate that the Finns did not expect much from the Canadians in the beginning, as they did not know whether they would be good coders or not. However, noticing the good work done by the Canadians the trust towards them started to rise. On the other hand, the Canadians had high expectations of the Finns, as they had met the Finnish instructor, the PO, and two experienced students, giving them a possibly false sense of trust towards the Finns. However, when the project proceeded it turned out that not all the Finnish students were as experienced and committed as the two visiting Canada, and the teams experienced some problems, and thus the trust started to deteriorate over time.

Interestingly however, a team which had already established trust became stronger when faced with difficulties:

*Because of the good communication and team trust that we build up, I think those sort of problems actually build*

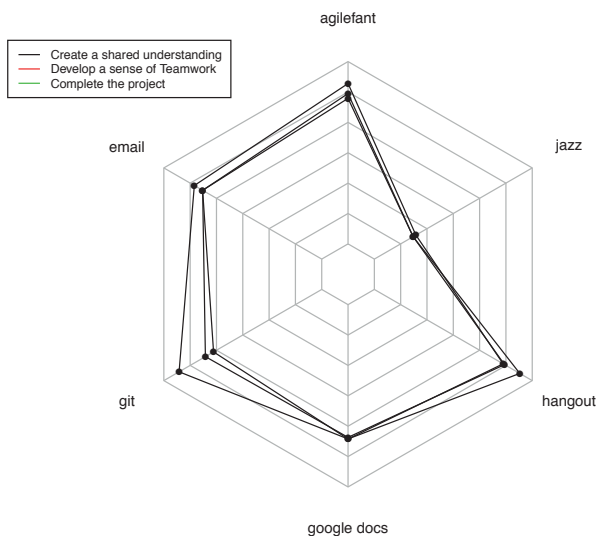


Fig. 2: Perceived usefulness of tools

*up the trust more, which is good.*

— A Canadian student

3) *Maintaining Trust through Awareness:* Daily Scrum meetings scheduled twice a week created visibility of team progress. Some of the students even commented that these meeting created a kind of pressure for the students to work between the meetings and to not appear in the meeting empty handed. By using the backlog management tool Agilefant each team was able to track all its assigned stories, their progress and each member's contribution to their development. Thus, Scrum seems to provide both practices (Daily Scrums) and tools (sprint and product backlogs) for maintaining trust through creating awareness.

#### D. Tool Usage for Distributed Collaboration

By experimenting with a number of collaboration tools to follow the Scrum's prescribed Daily Scrum meetings and frequent integration of code, students reported that one of their learning was that after this course they would be able to identify a suitable combination of communication tools for the many different purposes of collaboration in a distributed project, e.g., for synchronous and asynchronous communication, version control, and backlog management. Other than those suggested by the instructors, the students also experimented with tools such as Flowdock (<http://www.flowdock.com>) when they were looking for a common place to track all communication inside their team, such as discussions, documents and progress information. Another team used Google Docs to share awareness about project progress.

*[I would take] Google Hangout or some other videoconferencing software, then some chat program, preferably something where the whole team can be at the same time, but that enables also pair discussions, and as a third [tool] email, which is good for some things.*

— A Finnish student

*Now I understand better the differences between the different communication tools and would be able to do better decisions.*

— A Finnish student

In the post-course survey we asked the students to evaluate the usefulness of different tools they used for creating a shared understanding in the project, for developing a sense of teamwork, and for completing the project. Figure 2 shows the results. Google Hangout was considered the best tool for creating sense of teamwork, closely followed by Agilefant, email, Google Docs and git. For creating a shared understanding git, the configuration management system, was considered the best tool closely followed by Agilefant, Google Hangout, email and Google Docs. For completing the project, the most important tools were Google Hangout and Agilefant, closely followed by email, git and Google Docs. Surprisingly, the development environment Jazz was considered the least useful tool for all purposes, having significantly lower ratings than the other tools, which was possibly due to its overlapping functionality with Agilefant.

#### E. Culture

In Canada several students had varied cultural backgrounds, whereas in Finland all participants had a Finnish background. All practices supporting distributed communication and teamwork were used by the students in their inter-cultural collaboration. It seemed that the Finns and Canadians did not experience many cultural differences between the countries:

*I learned that between Finland and Canada there is not a big cultural difference. Even though we talked about it in the beginning, at least I did not notice it. (...) Somehow it felt that it did no matter, it was more like the personalities that mattered.*

— A Finnish student

*I don't think any cultural problems really came up. I was pretty comfortable talking to the Finns. (...) They are really not all that different. I mean, I guess they may not smile as much. (...) I actually think that the biggest cultural difference on our team was [a student] being from China and I think that with his cultural language barrier was actually more significant, even though he was located here, than any cultural difference between us and the Finns.*

— A Canadian student

One noted difference, however, was contrasting communication styles between the countries. The Finns were brief and direct, which at times came off as being rude. The Canadians on the other hand, incorporated a lot of small talk in to the conversation. Due to these differences, the Finns had difficulty understanding the Canadians' true opinions.

*I noticed that in demos they [Canadians] just talked and talked, and the Finns listened and said what we had to say and nothing extra. We were more like listeners and they talked. (...) I understood that this must have felt strange for the Canadians in the beginning and they had thought that we are boring people, but maybe they had figured it out that it is just that way in Finland.*

— A Finnish student

*I felt that in Canada, at least part of them, were afraid of saying aloud their opinions. (...) I would have hoped for more Finnish style of directness, it would have made it easier to make decisions.*

— Finnish student

Even though the students found some cultural differences they commented that the bigger differences were often in personalities of individuals and in their style of communication. All the interviewed students were very happy with the inter-cultural communication, and that during the course they had learned about another culture and got to know new people. Many of them actually wished after the course that there had been even more communication and collaboration between the countries, so that they would have gotten to know even more Canadians / Finns better, not only the ones they had been, e.g., pair programming with.

*[The best thing] was the team work with them [Canadians]. And then the technical variety of this product, and the variety in these people working with it.*

— A Finnish student

*The Finns were great. It was nice knowing people from Finland (...) I really haven't had opportunity to work with people in another country before, so that was my favorite part of it.*

— A Canadian student

## VI. CONCLUSIONS

In this paper we presented the course design using Scrum adapted to distributed environment following industry best practices and tools to support collaboration. This is to our knowledge the first GSE course to adopt such a combination of agile methodologies and best practices. As previously reported GSE courses have mainly used plan-driven methods, this course using distributed Scrum and our assessment of students expectations and learning are unique.

We also employed a mixed-method approach to assess the learning and the role played by distributed Scrum in teaching GSE skills of distributed communication and teamwork, using tools for distributed collaboration, building and maintaining trust, as well as inter-cultural collaboration. By analyzing data from before, during and after the course, we were able to discuss how by using practices of distributed Scrum students learned these skills. We discussed evidence of students' learning in the form of their reported (1) challenges in applying these skills, (2) strategies to overcome the challenges and (3) effectiveness of these strategies. We conclude that distributed Scrum augmented with GSE best practices is a suitable and important paradigm to consider in GSE course instructional design.

## REFERENCES

- [1] K. Schwaber and M. Beedle, *Agile Soft. development with Scrum*. Prentice-Hall, 2002.
- [2] K. Beck, "Embracing change with extreme programming," *Computer*, vol. 32, pp. 70–77, 1999.
- [3] M. Paasivaara, S. Durasiewicz, and C. Lassenius, "Using Scrum in a globally distributed project: A case study," *Soft. Process Improvement and Practice*, vol. 13, no. 6, pp. 527–544, 2008.
- [4] E. Hossain, M. A. Babar, and H.-y. Paik, "Using scrum in global soft. development: A systematic literature review," in *Proc. of the 4th IEEE Intl. Conf. on Global Soft. Eng.*, 2009, pp. 175–184.
- [5] G. Hanssen, D. Smite, and N. Moe, "Signs of agile trends in global soft. eng. research: A tertiary study," in *Intl. Conf. on Global Soft. Eng. Workshop (ICGSEW)*, 2011, pp. 17–23.
- [6] D. Damian, A. Hadwin, and B. Al-Ani, "Instructional design and assessment strategies for teaching global software development: a framework," in *Proc. of the 28th Intl. Conf. on Soft. Eng.*, 2006, pp. 685–690.
- [7] M. Monasor, A. Vizcaíno, M. Piattini, and I. Caballero, "Preparing students and engineers for global soft. development: A systematic review," in *Intl. Conf. on Global Soft. Eng.*, 2010, pp. 177–186.
- [8] L. Fortaleza, T. Conte, S. Marczak, and R. Prikladnicki, "Towards a gse intl. teaching network: Mapping global soft. eng. courses," in *Collaborative Teaching of Globally Distributed Soft. Development Workshop (CTGDSD)*, 2012, june 2012, pp. 1–5.
- [9] C. Scharff, O. Gotel, and V. Kulkarni, "Transitioning to distributed development in students' global soft. development projects: The role of agile methodologies and end-to-end tooling," in *5th Intl. Conf. on Soft. Eng. Advances (ICSEA)*, 2010, pp. 388–394.
- [10] C. Scharff, "Guiding global software development projects using scrum and agile with quality assurance," in *24th IEEE-CS Conf. on Soft. Eng. Education and Training (CSEET)*, 2011, pp. 274–283.
- [11] C. Scharff, S. Heng, and V. Kulkarni, "On the difficulties for students to adhere to scrum on global soft. development projects: Preliminary results," in *Collaborative Teaching of Globally Distributed Soft. Development Workshop (CTGDSD)*, 2012, pp. 25–29.
- [12] I. Cavrak, M. Orlic, and I. Crnkovic, "Collaboration patterns in distributed soft. development projects," in *34th Intl. Conf. on Soft. Eng. (ICSE)*, 2012, pp. 1235–1244.
- [13] I. Crnkovic, I. Bosnic, and M. Zagar, "Ten tips to succeed in global soft. eng. education," in *34th Intl. Conf. on Soft. Eng. (ICSE)*, 2012, pp. 1225–1234.
- [14] M. Nordio, R. Mitin, and B. Meyer, "Advanced hands-on training for distributed and outsourced soft. eng.," in *32nd Intl. Conf. on Soft. Eng. (ICSE)*, vol. 1, 2010, pp. 555–558.
- [15] A. Mockus and J. D. Herbsleb, "Challenges of global software development," *7th Intl. Symp. on Soft. METRICS*, pp. 182–184, 2001.
- [16] I. Richardson, S. Moore, D. Paulish, V. Casey, and D. Zage, "Globalizing software development in the local classroom," in *20th Conf. on Soft. Eng. Education Training (CSEET)*, 2007, pp. 64–71.
- [17] K. Swigger, R. Brazile, F. Serce, G. Dafoulas, F. Alpaslan, and V. Lopez, "The challenges of teaching students how to work in global soft. teams," in *Transforming Eng. Education: Creating Interdisciplinary Skills for Complex Global Environments*, 2010, pp. 1–30.
- [18] Y. Cai and W. Baelen, "On the development of pedagogical materials for globally distributed software engineering," in *Proc of Collaborative Teaching of Globally Distributed Software Development - Community Building Workshop*, 2012.
- [19] J. D. Herbsleb and R. E. Grinter, "Splitting the Organization and Integrating the Code: Conway's Law Revisited," in *Proceedings of the 21st International Conference on Software Engineering*. New York, NY, USA: ACM, 1999, pp. 85–95.
- [20] G. Spreitzer and A. Mishra, "Giving up control without losing control - Trust and its substitutes' effects on managers' involving employees in decision making," *Group & Organization Management*, vol. 24 (2), pp. 155–187, 1999.
- [21] M. Hoegl and H. Gemuenden, "Teamwork quality and the success of innovative projects: A theoretical concept and empirical evidence," *Organization Science*, vol. 12 (4), pp. 435–449, 2001.
- [22] M. Hoegl, K. Weinkauff, and H. Gemuenden, "Interteam coordination, project commitment, and teamwork in multiteam R&D projects: A longitudinal study," *Organization Science*, vol. 15 (1), pp. 38–55, 2004.
- [23] D. Damian, C. Lassenius, M. Paasivaara, A. Borici, and A. Schroter, "Teaching a globally distributed project course using Scrum practices," in *Collaborative Teaching of Globally Distributed Soft. Development Workshop (CTGDSD)*, 2012, june 2012, pp. 30–34.
- [24] M. Paasivaara and C. Lassenius, "Collaboration practices in global inter-organizational soft. development projects," *Soft. Process Improvement and Practice*, no. 4 (8), pp. 183–199, 2003.
- [25] T. Jick, "Mixing qualitative and quantitative methods: Triangulation in action," *Administrative Science Quarterly*, vol. 24 (4), 1979.
- [26] L. Freeman, "A set of measures of centrality based on betweenness," *Sociometry*, vol. 40, pp. 35–41, 1977.
- [27] V. D. Blondel, J.-L. Guilleme, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 10, p. 1000, 2008.